Running LISF 7.8 on a Linux system

James Geiger

Version 1, 10 Nov 2026

1. Overview

This document provides some instructions for compiling and running LISF (LDT, LIS, and LVT) on a generic Linux-based system.

More instructions are available at https://nasa-lis.github.io/LISF/.

2. Initial set up

NOTE	The default shell in Linux is Bash, and your account will contain a simple Bash *.profile* login script. The rest of this document assumes that you are using Bash.
NOTE	Many HPC systems use Environment Modules (the `module` command) to set compilers, tools, environment variables, etc. See https://modules.readthedocs.io/en/stable/index.html for more information. If you do not have Environment Modules, you may rewrite the contents of the provided *lisf_7.8_intel_2025.3.2* modulefile into a Bash script and `source` the script every time you log in. The rest of this document assumes that you have Environment Modules.

2.1. Initial set up of your environment

I have packaged all the required modules and environment variables needed to compile and run LISF into a custom modulefile. Please read the *Creating a custom modulefile* document (https://nasa-lis.github.io/LISF/creating_a_custom_modulefile/creating_a_custom_modulefile.html) and follow the instructions for setting up your Bash *.profile* file. See Section "Recommended files" for my recommended Bash *.profile* file, *.bashrc* file, and custom modulefile. Use the provided custom modulefile, *lisf_7.8_intel_2025.3.2*, as a basis for your own modulefile, make the appropriate changes for your Linux system, and place it into your *privatemodules* directory.

You may name your custom modulefile whatever you like. The rest of this document will use the name *lisf_7.8_intel_2025.3.2*.

3. Daily use

Here is the one step that must be performed every time that you log onto your system.

3.1. Load custom modulefile

Load the custom modulefile that sets up the LIS development environment. Run

```
% module load lisf_7.8_intel_2025.3.2
```

WARNING	I do **not** reccommend loading the modulefile automatically when logging in. Also, load only what is needed and unload when done. Loading everything is

> convenient, but it eventually leads to difficult-to-debug conflicts.

4. Accessing the LISF source code

There are two ways to access the LISF source code: via our NASA-LIS/LISF GitHub repository or via our LIS website.

4.1. GitHub

If you are doing software development, then use GitHub.

Please go to https://nasa-lis.github.io/LISF. Then follow the instructions in *LISF Working with GitHub Guide*.

The *master* branch contains the latest development to LISF.

4.2. LIS website

If you wish to use a stable version of LISF, then use the LIS website.

Please go to https://lis.gsfc.nasa.gov. Click on *Source*. Click on *LIS Releases*. There you will find the LISF public release tar-balls containing the LISF source code.

This is also available in our NASA-LIS/LISF GitHub repository under Releases.

The *support/lisf-public-7.8* branch is the latest stable release of LISF.

5. Compiling LDT

More instructions are available in the "LDT Users Guide" (https://nasa-lis.github.io/LISF/LDT_users_guide/LDT_users_guide.html).

5.1. Step 1

To compile LDT, first run the *configure* script.

It is recommended to compile LDT serially (without MPI support). When prompted, please select "serial":

```
Parallelism (0-serial, 1-dmpar, default=1): 0
```

Recommended settings

```
Parallelism (0-serial, 1-dmpar, default=0): 0
Optimization level (-3=strict checks with warnings, -2=strict checks, -1=debug,
0,1,2,3, default=2): 2
```

```
Assume little/big_endian data format (1-little, 2-big, default=2): 2
Use GRIBAPI/ECCODES? (0-neither, 1-gribapi, 2-eccodes, default=2): 2
NETCDF version (3 or 4, default=4)?: 4
NETCDF use shuffle filter? (1-yes, 0-no, default = 1): 1
NETCDF use deflate filter? (1-yes, 0-no, default = 1): 1
NETCDF use deflate level? (1 to 9-yes, 0-no, default = 1): 1
Use HDF4? (1-yes, 0-no, default=1): 1
Use HDF5? (1-yes, 0-no, default=1): 1
Use HDFEOS? (1-yes, 0-no, default=1): 1
Enable GeoTIFF support? (1-yes, 0-no, default=1): 1
Enable LIBGEOTIFF support? (1-yes, 0-no, default=1): 1
Include date/time stamp history? (1-yes, 0-no, default=1): 1
Use PIO? (1-yes, 0-no, default=0): 1
```

5.2. Step 2

Run the *compile* script.

```
./compile -j 16
```

Please see Section "Notes regarding the *compile* script" for additional information regarding running the *compile* script.

Please see Section "Common warnings" for additional information regarding compiling/linking LDT.

6. Compiling LIS

More instructions are available in the "LIS Users Guide" (https://nasa-lis.github.io/LISF/LIS_users_guide/LIS_users_guide.html).

6.1. Step 1

If you wish to compile LIS with its default features enabled, then you may proceed to Step 2, below.

Otherwise, please see the "LIS Users Guide" for more information on customizing the build via LIS plugins.

Example lis/make/user.cfg file, for reference:

```
VIC.4.1.1: Off
VIC.4.1.2: Off
```

6.2. Step 2

To compile LIS, first run the *configure* script.

Recommended settings

```
Parallelism (0-serial, 1-dmpar, default=1): 1
Optimization level (-3=strict checks with warnings, -2=strict checks, -1=debug,
0,1,2,3, default=2): 2
Assume little/big_endian data format (1-little, 2-big, default=2): 2
Use GRIBAPI/ECCODES? (0-neither, 1-gribapi, 2-eccodes, default=2): 2
Enable AFWA-specific grib configuration settings? (1-yes, 0-no, default=0): 0
Use NETCDF? (1-yes, 0-no, default=1): 1
NETCDF version (3 or 4, default=4): 4
NETCDF use shuffle filter? (1-yes, 0-no, default = 1): 1
NETCDF use deflate filter? (1-yes, 0-no, default = 1): 1
NETCDF use deflate level? (1 to 9-yes, 0-no, default = 1): 1
Use HDF4? (1-yes, 0-no, default=1): 1
Use HDF5? (1-yes, 0-no, default=1): 1
Use HDFEOS? (1-yes, 0-no, default=1): 1
Use MINPACK? (1-yes, 0-no, default=0): 0
Use LIS-CRTM? (1-yes, 0-no, default=0): 0
Use LIS-CMEM? (1-yes, 0-no, default=0): 0
Use LIS-LAPACK? (0-no, 1-mkl, 2-lapack/blas, 3-lapack/refblas, default=0): 0
Use PETSc? (1-yes, 0-no, default=0): 0
Enable USAF LIS75 SM DA codes? (1-yes, 0-no, default=0): 0
Use PIO? (1-yes, 0-no, default=0): 1
```

6.3. Step 3

Run the *compile* script.

```
./compile -j 16
```

Please see Section "Notes regarding the *compile* script" for additional information regarding running the *compile* script.

Please see Section "Common warnings" for additional information regarding compiling/linking LIS.

7. Compiling LVT

More instructions are available in the "LVT Users Guide" (https://nasa-lis.github.io/LISF/LVT_users_guide/LVT_users_guide.html).

7.1. Step 1

To compile LVT, first run the *configure* script.

Recommended settings

```
Optimization level (-3=strict checks with warnings, -2=strict checks, -1=debug,
0,1,2,3, default=2): 2
Assume little/big_endian data format (1-little, 2-big, default=2): 2
Use GRIBAPI/ECCODES? (1-gribapi, 2-eccodes, default=2): 2
Use NETCDF? (1-yes, 0-no, default=1): 1
NETCDF version (3 or 4, default=4): 4
NETCDF use shuffle filter? (1-yes, 0-no, default = 1): 1
NETCDF use deflate filter? (1-yes, 0-no, default = 1): 1
NETCDF use deflate level? (1 to 9-yes, 0-no, default = 1): 1
Use HDF4? (1-yes, 0-no, default=1): 1
Use HDF5? (1-yes, 0-no, default=1): 1
Use HDFEOS? (1-yes, 0-no, default=1): 1
Enable AFWA-specific grib configuration settings? (1-yes, 0-no, default=0): 0
Enable GeoTIFF support? (1-yes, 0-no, default=1): 1
Use MATLAB support? (1-yes, 0-no, default=0): 0
Use PIO? (1-yes, 0-no, default=0): 1
```

7.2. Step 2

Run the *compile* script.

```
./compile -j 16
```

Please see Section "Notes regarding the *compile* script" for additional information regarding running the *compile* script.

Please see Section "Common warnings" for additional information regarding compiling/linking LVT.

8. Notes regarding the *compile* script

The `-j 16` switch causes the *compile* script to invoke `make` with its `-j 16` switch. This has been observed to speed up compiling.

The `-j` switch is optional. Run `./compile -h` for more information.

Compiling LISF writes a lot of messages to stdout (the screen). Running

```
./compile -j 16 >make.log 2>&1
```

will write the messages into a file named *make.log*. This file is easier to read (to find error

messages), and it is easier to share (better than making screenshots).

9. Common warnings

When linking LDT, LIS, and LVT, you may get warnings such as:

Common linking warnings

```
ld: warning: libjpeg.so.8, needed by /usr/lib64/libtiff.so, may conflict with
libjpeg.so.62

ld: warning: size of symbol `$data_init$lis_mpimod_' changed from 4 in
AGRMET_fldbld_gfs.o to 8 in LIS_mpiMod.o

ld: warning: size of symbol `$data_init$errormessagelength_mod_' changed from 4 in
driver_ascii_mod.o to 8 in errormessagelength_mod.o

ld: warning: size of symbol `$data_init$veg_param_' changed from 4 in
ecosse_prepare_mod.o to 8 in veg_param_mod.o
```

Usually, these may be ignored.

10. Running LIS

First, establish a working directory to run LIS in. This will be referred to as `$WORKDIR`. Then go into that working directory:

```
% cd $WORKDIR
```

Then transfer all required data to run LIS into here. Please note that your `$WORKDIR` may be scrubbed. So archive any important input and output data elsewhere.

Many HPC systems use a job manager like SLURM to manage jobs running on the compute nodes. See "Recommended files" for a sample submission script. Save the script as *lis.job* and submit it via:

```
% sbatch lis.job
```

NOTE

If your system does not use a job/queue manager, then you may run the *lis.job* file as a Bash script. But first, you must provide values for any variables that would have been set by SLURM; e.g.,

```
SLURM_NTASKS=234 bash ./lis.job
```

11. Recommended files

11.1. Custom modulefile

Use this custom modulefile as the basis for your own Linux system and place it into your *privatemodules* directory.

```
#%Module1.0#####################################################################

proc ModulesHelp { } {
    puts stderr "\t[module-info name] - loads the LISF_7_8_INTEL_2025_3_2 env"
    puts stderr ""
    puts stderr "This is for use on NCCS' discover system running SLES 15.4."
    puts stderr ""
    puts stderr "\tThe following env variables are set:"
    puts stderr "\t\tDEV_ENV"
    puts stderr "\t\tLIS_ARCH"
    puts stderr "\t\tLIS_SPMD"
    puts stderr "\t\tLIS_FC"
    puts stderr "\t\tLIS_CC"
    puts stderr "\t\tLIS_PIO_IN_ESMF"
    puts stderr "\t\tLIS_RPC"
    puts stderr "\t\tLIS_OPENJPEG"
    puts stderr "\t\tLIS_ECCODES"
    puts stderr "\t\tLIS_NETCDF"
    puts stderr "\t\tLIS_HDF4"
    puts stderr "\t\tLIS_HDFEOS"
    puts stderr "\t\tLIS_HDF5"
    puts stderr "\t\tLIS_MODESMF"
    puts stderr "\t\tLIS_LIBESMF"
    puts stderr "\t\tLIS_MINPACK"
    puts stderr "\t\tLIS_CRTM"
    puts stderr "\t\tLIS_CRTM_PROF"
    puts stderr "\t\tLIS_CMEM"
    puts stderr "\t\tLIS_LAPACK"
    puts stderr "\t\tLIS_PETSC"
    puts stderr "\t\tLDT_ARCH"
    puts stderr "\t\tLDT_FC"
    puts stderr "\t\tLDT_CC"
    puts stderr "\t\tLDT_PIO_IN_ESMF"
    puts stderr "\t\tLDT_RPC"
    puts stderr "\t\tLDT_OPENJPEG"
    puts stderr "\t\tLDT_ECCODES"
    puts stderr "\t\tLDT_NETCDF"
    puts stderr "\t\tLDT_HDF4"
    puts stderr "\t\tLDT_HDFEOS"
    puts stderr "\t\tLDT_HDF5"
    puts stderr "\t\tLDT_MODESMF"
    puts stderr "\t\tLDT_LIBESMF"
```

```
    puts stderr "\t\tLDT_GDAL"
    puts stderr "\t\tLDT_FORTRANGIS"
    puts stderr "\t\tLDT_LIBGEOTIFF"
    puts stderr "\t\tLVT_ARCH"
    puts stderr "\t\tLVT_FC"
    puts stderr "\t\tLVT_CC"
    puts stderr "\t\tLVT_PIO_IN_ESMF"
    puts stderr "\t\tLVT_RPC"
    puts stderr "\t\tLVT_OPENJPEG"
    puts stderr "\t\tLVT_ECCODES"
    puts stderr "\t\tLVT_NETCDF"
    puts stderr "\t\tLVT_HDF4"
    puts stderr "\t\tLVT_HDFEOS"
    puts stderr "\t\tLVT_HDF5"
    puts stderr "\t\tLVT_MODESMF"
    puts stderr "\t\tLVT_LIBESMF"
    puts stderr "\t\tLVT_GDAL"
    puts stderr "\t\tLVT_FORTRANGIS"
    puts stderr ""
    puts stderr "\tThe following modules are loaded:"
    puts stderr "\t\tcomp/gcc/16.1.0"
    puts stderr "\t\tcomp/intel/2025.3.2"
    puts stderr "\t\tmpi/impi/2021.17"
    puts stedrr "\t\tgit/2.49.0"
    puts stderr ""
}

conflict comp mpi

module-whatis   "loads the [module-info name] environment"

set modname     [module-info name]
set modmode     [module-info mode]

# icpx depends on headers provided by g++
module load comp/gcc/16.1.0
module load comp/intel/2025.3.2
module load mpi/impi/2021.17
module load git/2.49.0

set   def_lis_rpc         /usr/lib64/libtirpc.so
set   def_lis_hdf5        /discover/nobackup/projects/lis/libs/sles-
15.4/hdf5/1.14.6_intel-2025.3.2
set   def_lis_netcdf      /discover/nobackup/projects/lis/libs/sles-
15.4/netcdf/4.10.0_intel-2025.3.2
set   def_lis_openjpeg    /discover/nobackup/projects/lis/libs/sles-
15.4/openjpeg/2.3.0-150000.3.13.1.x86_64/usr
set   def_lis_eccodes     /discover/nobackup/projects/lis/libs/sles-
15.4/eccodes/2.47.0_intel-2025.3.2
set   def_lis_hdf4        /discover/nobackup/projects/lis/libs/sles-
15.4/hdf4/4.3.1_intel-2025.3.2
```

```
set   def_lis_hdfeos      /discover/nobackup/projects/lis/libs/sles-
15.4/hdfeos2/3.0_intel-2025.3.2
set   def_lis_modesmf     /discover/nobackup/projects/lis/libs/sles-
15.4/esmf/8.9.1_intel-2025.3.2_impi-2021.17/mod/modO/Linux.intel.64.intelmpi.default
set   def_lis_libesmf     /discover/nobackup/projects/lis/libs/sles-
15.4/esmf/8.9.1_intel-2025.3.2_impi-2021.17/lib/libO/Linux.intel.64.intelmpi.default
set   def_lvt_proj        /discover/nobackup/projects/lis/libs/sles-
15.4/proj/9.8.1_intel-2025.3.2
set   def_ldt_libgeotiff  /discover/nobackup/projects/lis/libs/sles-
15.4/geotiff/1.7.4_intel-2025.3.2
set   def_lvt_gdal        /discover/nobackup/projects/lis/libs/sles-
15.4/gdal/3.13.1_intel-2025.3.2
set   def_lvt_fortrangis  /discover/nobackup/projects/lis/libs/sles-
15.4/fortrangis/3.0-2_intel-2025.3.2
set   def_lis_petsc       /discover/nobackup/projects/lis/libs/sles-
15.4/petsc/3.21.6_intel-2025.3.2_impi-2021.17
set   def_lis_minpack     /discover/nobackup/projects/lis/libs/minpack/intel_11_1_038
set   def_lis_crtm        /discover/nobackup/projects/lis/libs/JCSDA_CRTM/REL-
2.0.2.Surface-rev_intel_18_0_3_222
set   def_lis_crtm_prof
/discover/nobackup/projects/lis/libs/CRTM_Profile_Utility/intel_18_0_3_222
set   def_lis_cmem        /discover/nobackup/projects/lis/libs/LIS-
MEM/intel_18_0_3_222
set   def_lis_lapack
/discover/nobackup/projects/lis/libs/lapack/3.6.0_intel_14_0_3_174

setenv   DEV_ENV    LISF_7_8_INTEL_2025_3_2

setenv   LIS_ARCH        linux_ifx
setenv   LIS_SPMD        parallel
setenv   LIS_FC          mpiifort
setenv   LIS_CC          mpicc
setenv   LIS_PIO_IN_ESMF 1
setenv   LIS_RPC         $def_lis_rpc
setenv   LIS_OPENJPEG    $def_lis_openjpeg
setenv   LIS_ECCODES     $def_lis_eccodes
setenv   LIS_NETCDF      $def_lis_netcdf
setenv   LIS_HDF4        $def_lis_hdf4
setenv   LIS_HDFEOS      $def_lis_hdfeos
setenv   LIS_HDF5        $def_lis_hdf5
setenv   LIS_MODESMF     $def_lis_modesmf
setenv   LIS_LIBESMF     $def_lis_libesmf
setenv   LIS_MINPACK     $def_lis_minpack
setenv   LIS_CRTM        $def_lis_crtm
setenv   LIS_CRTM_PROF   $def_lis_crtm_prof
setenv   LIS_CMEM        $def_lis_cmem
setenv   LIS_LAPACK      $def_lis_lapack
setenv   LIS_PETSC       $def_lis_petsc

setenv   LDT_ARCH        linux_ifx
setenv   LDT_FC          mpiifort
```

```
setenv   LDT_CC          mpicc
setenv   LDT_PIO_IN_ESMF 1
setenv   LDT_RPC         $def_lis_rpc
setenv   LDT_OPENJPEG    $def_lis_openjpeg
setenv   LDT_ECCODES     $def_lis_eccodes
setenv   LDT_NETCDF      $def_lis_netcdf
setenv   LDT_HDF4        $def_lis_hdf4
setenv   LDT_HDFEOS      $def_lis_hdfeos
setenv   LDT_HDF5        $def_lis_hdf5
setenv   LDT_MODESMF     $def_lis_modesmf
setenv   LDT_LIBESMF     $def_lis_libesmf
setenv   LDT_GDAL        $def_lvt_gdal
setenv   LDT_FORTRANGIS  $def_lvt_fortrangis
setenv   LDT_LIBGEOTIFF  $def_ldt_libgeotiff


setenv   LVT_ARCH        linux_ifx
setenv   LVT_FC          mpiifort
setenv   LVT_CC          mpicc
setenv   LVT_PIO_IN_ESMF 1
setenv   LVT_RPC         $def_lis_rpc
setenv   LVT_OPENJPEG    $def_lis_openjpeg
setenv   LVT_ECCODES     $def_lis_eccodes
setenv   LVT_NETCDF      $def_lis_netcdf
setenv   LVT_HDF4        $def_lis_hdf4
setenv   LVT_HDFEOS      $def_lis_hdfeos
setenv   LVT_HDF5        $def_lis_hdf5
setenv   LVT_MODESMF     $def_lis_modesmf
setenv   LVT_LIBESMF     $def_lis_libesmf
setenv   LVT_GDAL        $def_lvt_gdal
setenv   LVT_FORTRANGIS  $def_lvt_fortrangis


prepend-path   LD_LIBRARY_PATH   "$def_lis_openjpeg/lib"
prepend-path   LD_LIBRARY_PATH   "$def_ldt_libgeotiff/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lvt_proj/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lvt_gdal/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lis_hdf4/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lis_hdf5/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lis_libesmf"
prepend-path   LD_LIBRARY_PATH   "$def_lis_netcdf/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lis_eccodes/lib"
prepend-path   LD_LIBRARY_PATH   "$def_lis_petsc/lib"
prepend-path   PATH   "$def_lis_netcdf/bin:$def_lis_eccodes/bin"
```

11.2. Sample SLURM submission script

Save this script as *lis.job*.

```
#!/bin/bash
#SBATCH --job-name OL-use-case
#SBATCH --ntasks=234
#SBATCH --qos=debug
#SBATCH --constraint=mil
#SBATCH --time=00:20:00
#SBATCH --account=s1189

ulimit -s unlimited

mpirun -n $SLURM_NTASKS ./LIS --file lis.config.laplata.noahmp401.hymap.4km.OL.use-
case >ol-use-case.err 2>&1
```

11.3. Recommended Bash *.profile* file

```
# .bash_profile

test -z "$PROFILEREAD" && . /etc/profile || true


# User specific environment and startup programs

ulimit -s unlimited

module use --append $HOME/privatemodules
```

11.4. Recommended *.bashrc* file

```
# .bashrc

# User specific aliases and functions
```

Appendix A: Libraries

A.1. ecCodes

Care must be taken when installing the ecCodes library.

The build process for ecCodes embeds paths into the library, so you must make sure to properly set the `-DCMAKE_INSTALL_PREFIX` option to `cmake`. After you run `cmake`, run `make`, `make check`, and finally `make install`. If you do not specify the CMAKE_INSTALL_PREFIX and use `make install`, then the library will not be able find its definition files.

If you do not install ecCodes according to their instructions, then you will get error messages like

this when running LISF:

```
ecCodes Version:       2.7.0
Definition files path: /h/LIBRARIES/share/grib_api/definitions
ECCODES ERROR   :  grib_parser_include: Could not resolve 'section.0.def' (included in
/gpfs/h/LIBRARIES/share/grib_api/definitions/grib2/boot.def)
```

If you must compile the ecCodes library in one place/system and then copy the files into another place/system, then you must make sure to set the `ECCODES_DEFINITION_PATH` and the `ECCODES_SAMPLES_PATH` environment variables. Set the `ECCODES_DEFINITION_PATH` environment variable to the full path that points to *share/eccodes/definitions*. Set the `ECCODES_SAMPLES_PATH` environment variable to the full path that points to *share/eccodes/samples*. See the ecCodes FAQ for more information: https://confluence.ecmwf.int/pages/viewpage.action?pageId=143041487